Voodoo Dough Boys

The Calculatorinator
Software Requirements Specifications

Version 1.0

Revision History

Date	Version	Description	Author
03/03/26	1.0	Filled out the Introduction (minus References) and also the Overall Description	Vu Nguyen
09/03/26	1.0	Filled in UML diagram	Micah Lehman
12/03/26	1.0	Filled out section 3 and 4 of Specific Requirements	Joseph Ubanda
12/03/26	1.0	Filled in more of the definitions acronyms and abbreviations, and the purpose	La`Nale Garry
14/03/26	1.0	Add the UML diagram to section 3	Vu Nguyen Joseph Ubanda

Table of Contents

1. Introduction 4
   - 1.1 Purpose 4
   - 1.2 Scope 4
   - 1.3 Definitions, Acronyms, and Abbreviations 4
   - 1.4 References 4
   - 1.5 Overview 4

2. Overall Description 5
   - 2.1 Product perspective 5
     - 2.1.1 System Interfaces 5
     - 2.1.2 User Interfaces 5
     - 2.1.3 Hardware Interfaces 5
     - 2.1.4 Software Interfaces 5
     - 2.1.5 Communication Interfaces 5
     - 2.1.6 Memory Constraints 5
     - 2.1.7 Operations 5
   - 2.2 Product functions 5
   - 2.3 User characteristics 5
   - 2.4 Constraints 5
   - 2.5 Assumptions and dependencies 5
   - 2.6 Requirements subsets 6

3. Specific Requirements 6
   - 3.1 Functionality 6
     - 3.1.1 Tokenization of user input: 6
     - 3.1.2 Support for +, -, *, /, %, ** 6
     - 3.1.3 Parentheses handling 6
     - 3.1.4 Check for Syntax and division by zero 6
     - 3.1.5 Unary operators 6
     - 3.1.6 User-interface 6
     - 3.1.7 Numeric Constants 6
   - 3.2 Use-Case Specifications 6
   - 3.3 Supplementary Requirements 7
     - 3.3.1 Robust Error Handling 7
     - 3.3.2 Floating Point Support 7

4. Classification of Functional Requirements 8

5. Appendices 8

Software Requirements Specifications

1. Introduction

The SRS (Software Requirement Specifications) will act as a guideline for all the requirements, FR (Functional Requirements) and NFR (Non Functional Requirements), in the project. Specifics about each requirement and UC (Use Cases) will be documented as well as constraints about the project. A use case diagram will also be provided in the SRS.

1.1 Purpose

The purpose of the SRS is to act as a guide to the developers about what things will be required when they begin programming “The Calculatorinator”. The document will have each requirement of the project clearly recorded and give clarity to help with any confusion about the requirements for the developers. It will also show the NFR, FR, design constraints, and UC that the developers will need to know in order for a smoother development process for the final project.

1.2 Scope

The SRS will be used throughout the entire project to specify requirements for the developers and be updated frequently. The UC modeling will be based on the requirements in this document.

1.3 Definitions, Acronyms, and Abbreviations

SRS: Software Requirements Specification, which is this document
Functional Requirements (FR): Define the specific behaviors functions or services the system needs to provide
Non Functional Requirements (or NFR): Define system properties like security, reliability, performance, or usability.
Use Cases (UC):Descriptions for the ways a user can interact with the system to achieve a given goal.
Design constraints: Limitations on the designs and products created or the processes you use while making them.

1.4 References

1.5 Overview

The following will provide a more in depth explanation of each requirement in the SRS and a use case diagram will be provided for a visualization.

2. Overall Description

This project will take an expression from the user's input, evaluate that function, and then print out the correct value. When errors occur, the program will be able to tell the user what happened. This project will be implemented in C++ and managed with github.

2.1 Product perspective

2.1.1 System Interfaces

There will be no software interfaces for the project.

2.1.2 User Interfaces

The user interface will simply be a command line input where the user can type in an expression and the result will also be shown on the command line in the form of text.

2.1.3 Hardware Interfaces

There will be no hardware interfaces for this project.

2.1.4 Software Interfaces

There will be no software interfaces in this project since the project consists only of a program that runs using the command line.

2.1.5 Communication Interfaces

There will be no communication interfaces for this project.

2.1.6 Memory Constraints

The program will have a limit that is based off of the C++ programming language's integer limit which is 2^32 - 1 for positive integers and negative 2^32 for negative integers.

2.1.7 Operations

The Calculatorinator will be able to take an input with the following operators :

- + (addition)
- - (subtraction)
- * (multiplication)
- / (division)
- % (modulo)
- ** (exponentiation)
- + - unary operators

2.2 Product functions

The program should be able to take an equation that is entered in via the command line and provide the value of the expression. The project will be able to handle errors and tell the user which error has occurred.

2.3 User characteristics

This project is created for any user who wishes to have a simple expression evaluated and has basic knowledge of how to input the equation.

2.4 Constraints

This program should be implemented using C++ programming language and be able to run on common operating systems like Windows and Linux.

2.5 Assumptions and dependencies

The assumptions of the project will be provided below:

- The user may input an expression of any length

- The user can enter in any expression or an input that is not an expression
- The user enters the expression using the command line
- The program should compile and run properly

2.6 Requirements subsets

There are 3 different types of requirements in the SRS: Functional, Non-Functional, and Constraints. Functional requirements will be concrete things that need to be implemented into the program. Non-Functional requirements will be requests of how the program should behave given certain situations. Constraints will describe what the program will accept and the specifics about things needed when writing the program.

3. Specific Requirements

3.1 Functionality

3.1.1 Tokenization of user input:

The Program will tokenize the user input string into individual operands and operators.

*3.1.2 Support for +, -, *, /, %, ** :*

The program should support addition, subtraction, multiplication, division, modulo, and exponent operators.

3.1.3 Parentheses handling

The program will support the PEMDAS order of operations when evaluating the user's arithmetic expression.

3.1.4 Check for Syntax and division by zero

The program handles errors such as division by zero and syntax errors and then displays the corresponding error message to the user.

3.1.5 Unary operators

Support unary + and - within the user’s input expression.

3.1.6 User-interface

The user interface command line is where the user can enter the input expression and then see their results.

3.1.7 Numeric Constants

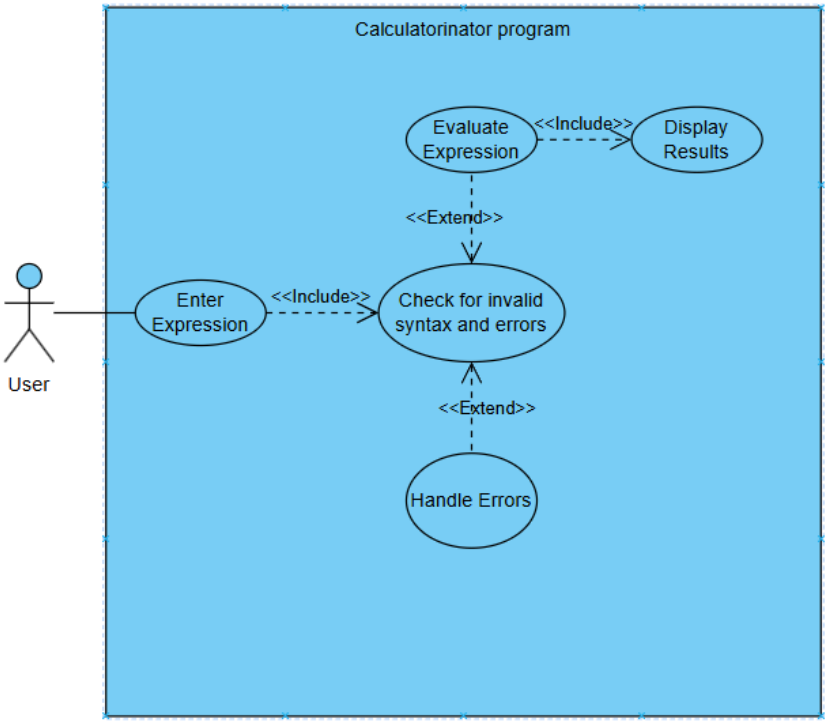
Recognize and calculate numeric constants within the expression.

3.2 Use-Case Specifications

Primary Actor: User

1. User enters an arithmetic string expression.
2. Check for invalid characters or errors in the expression.
3. If errors occur handle the error.
4. If no errors occur, evaluate the input expression.

5. Then display results.



3.3 Supplementary Requirements

3.3.1 Robust error handling
Inform the user of the error in the command line.

3.3.2 Floating point support
Allow for float number values to be used within the expression.

4. Classification of Functional Requirements

Functionality	Type
Tokenization of input strings	Essential
Support for +, -, *, /, %, **	Essential
Parentheses handling	Essential
Check for Syntax and division by zero	Essential
Unary operators	Essential
User-interface	Essential
Numeric Constants	Essential
Floating point support	Desirable
Robust error handling	Optional

5. Appendices

This section will be updated whenever any appendices are added to the SRS. As of the current version, there are no appendices.